previous listing, binding each `<select>` list value to a different property but reusing the same `Items` list for all of them.

Listing 18.6 Razor template to display a select element in three ways

```
@page
@model SelectListsModel
<select asp-for="Input.SelectedValue1"                    ❶
    asp-items="Model.Items"></select>                     ❶
<select asp-for="Input.SelectedValue2"                    ❷
    asp-items="Model.Items" size="4"></select>            ❷
<select asp-for="Input.MultiValues"
    asp-items="Model.Items"></select>                     ❸
```

❶ Creates a standard drop-down select list by binding to a standard property in asp-for

❷ Creates a single-select list box of height 4 by providing the standard HTML size attribute

❸ Creates a multiselect list box by binding to an IEnumerable property in asp-for

I hope you can see that the Razor for generating a drop-down `<select>` list is almost identical to the Razor for generating a multiselect `<select>` list. The Select Tag Helper takes care of adding the `multiple` HTML attribute to the generated output if the property it's binding to is an `IEnumerable`.

WARNING The `asp-for` attribute *must not* include the `Model.` prefix. The `asp-items` attribute, on the other hand, *must* include it if referencing a property on the `PageModel`. The `asp-items` attribute can also reference other C# items, such as objects stored in `ViewData`, but using a `PageModel` property is the best approach.

You've seen how to bind three types of select lists so far, but the one I haven't yet covered from figure 18.6 is how to display groups in your list boxes using `<optgroup>` elements. Luckily, nothing needs to change in your Razor code; you have to update only how you define your `SelectListItem`s.